This work is licensed under a Creative Commons license

Attribution-NonCommercial-NoDerivatives 4.0 International
(CC BY-NC-ND 4.0)

You are free to:

- **Share** – copy and redistribute the material in any medium or format.

Under the following terms:

- **Attribution** – You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- **NonCommercial** – You may not use the material for commercial purposes.
- **NoDerivatives** – If you remix, transform, or build upon the material, you may not distribute the modified material.

Surreptitious Software

Foundations of Software Protection and Reverse Engineering Resistance

Simone Aonzo, PhD

Last update: 2026-06-02T10:16:49

Surreptitious Software

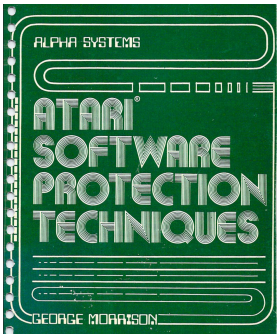
Software that *deliberately conceals* its internal logic to protect intellectual property (IP) and resist analysis [NC09]

Threat Model: assumes an attacker with full binary access and tooling

- **Obfuscation**: preserve functionality while degrading analyzability
  - Increase the *cost & time* of reverse engineering
  - Adaptive behavior to avoid observability
- **Tamperproofing**: detect or fail under unauthorized modification
  - Protect *integrity*
- **Watermarking**: embed identifying information
  - Enable *ownership claims*
- **Evasion**: detect runtime environments or analysis components
  - To change behavior accordingly

Trade-offs: performance (time), size (space), and maintainability

Atari Software Protection Techniques (1983)



Can any program be made totally uncopyable?
*This question gets a qualified **NO**.*
For most practical purposes, any software can be pirated.
No matter how complex the protection technique, there are people who can break it.
Any protection technique invented by man can be broken by man.

Software Protectors

Transform regular software into surreptitious software

- **Anti-piracy:** licensing and Digital Rights Management (DRM)
- Prevent adding unwanted functionalities
  - E.g., APIs for botting or cheating
- Protectors are complex systems requiring ongoing updates
  - They must keep up with the attackers
  - $\Longrightarrow$ They are often commercial software
- Integrated at source-level and in the build pipeline
  - More robust rather than applied as a black-box to a compiled binary

Famous Protectors:

- Denuvo 👑
  - Video: Reverse Engineering Denuvo in Hogwarts Legacy [link]
- VMProtect
- Themida
- Enigma
- Obsidium

The Art of Software Protection

Software Protection is the *"art"* of creating surreptitious software that uses a **synergy** between obfuscation, evasion, tamperproofing and watermarking.

The performances and maintainability/testing of such software are governed by a delicate **trade-off**: every extra layer of protection increases the cost to the defender as well as the attacker.

Assume that the security it provides will be breached by an attacker with sufficient **motivation**, **skill**, and **financial resources**
$\Longrightarrow$ move critical logic/secrets to server when possible

The ultimate goal is not to achieve everlasting security, but to maximize the attacker's effort while minimizing the defender's cost and impact on legitimate use

The Game Theory of Videogame Industry

Videogame revenue rule of thumb

$\approx$ 50% first 14 days
$\approx$ 30% over the next 3-10 weeks
$\implies$ $\approx$ 80% in the first 2 months

Economic interpretation

- Launch period: high information asymmetry and max price elasticity
- Marketing, pre-orders, and hype: front-loaded demand curve
- After this window: price drops and discounts
- $\implies$ publishers optimize time-to-crack, not absolute unbreakability
- $\implies$ *which protections best safeguard the launch window?*
- Attacker model: **Warez scene** AKA **The Scene**
  - Motivation: prestige and recognition
  - Less to none financial incentives
  - Members invest personal time
  - High skilled reverse engineers

Outline

Outline

Obfuscation

Obfuscation consists of transforming a program p into a semantically equivalent p', which is more difficult to analyze/understand

Obfuscated code can be larger and slower

- risk of introducing bugs
- difficult to test and debug

E.g., "Users have found that Denuvo, causes slowdown, crashes, and freezes in legally purchased copies of the game" `https://www.gamerevolution.com/news/409231-sonic-mania-plus-drm-protection-slowing-down-legitimate-copies`

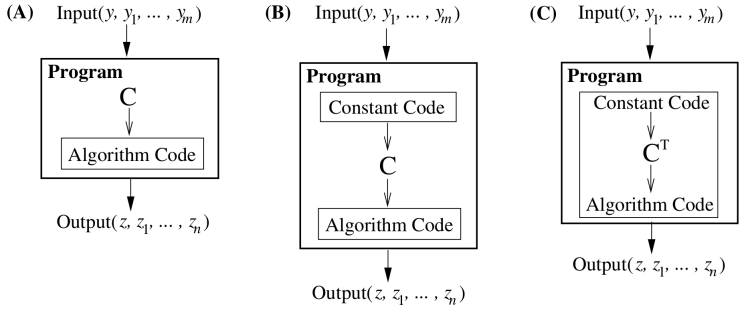
Obfuscation Classes

- **Data**: concealed values are reconstructed at runtime
- **Control-flow**: makes it difficult to understand which instructions are actually executed and whether they are relevant to the actual functioning of the program
  - *Intra-procedural*
  - *Inter-procedural*
- **External functions**: invoke (lib|sys)calls in unconventional ways
- **Anti-Disassembly**: introduce instructions that mislead static analysis
- **Packing**: hides the whole binary, unpacked at runtime

The synergistic composition of obfuscation techniques (layering and interaction effects) yields a program whose semantics and structure are far more difficult to analyze than any single technique alone

Outline

Data Obfuscation



- (a) both the constant C and the algorithm are in the clear
- (b) C is an *opaque constant*; i.e., C is replaced by code calculating C
- (c) C^T, i.e., C under transformation T, is created at runtime and the algorithm is modified to work with C^T
  - $\Longrightarrow$ C is never directly exposed in memory

Data Obfuscation (A) – Example

```
PROCESSENTRY32W pe = { 0 };
pe.dwSize = sizeof(pe);
BOOL found = FALSE;

if (Process32FirstW(snap, &pe)) {
    do {
        DWORD pid = pe.th32ProcessID;
        wprintf(L"%6lu  %s\n", pid, pe.szExeFile);
        if (_wcsicmp(pe.szExeFile, L"Procmon64.exe") == 0) {
            wprintf(L"Found! PID=%lu\n", pid);
            found = TRUE;
            break;
        }
    } while (Process32NextW(snap, &pe));
}
```

Data Obfuscation (B) – Example

```
uint8_t data[] = {
    0x12,0x30,0x2d,0x21,0x2f,0x2d,0x2c,0x74,0x76,0x6c,0x27,0x3a,0x27};
size_t len = sizeof(data) / sizeof(data[0]);
wchar_t* decoded = (wchar_t*) malloc((len + 1) * sizeof(wchar_t));
// [...]
for (size_t i = 0; i < len; i++) {
    decoded[i] = (wchar_t)(data[i] ^ 0x42);
}
decoded[len] = L'\0';
// [...]
if (Process32FirstW(snap, &pe)) {
    do {
        DWORD pid = pe.th32ProcessID;
        wprintf(L"%6lu  %s\n", pid, pe.szExeFile);
        if (_wcsicmp(pe.szExeFile, decoded) == 0) {
            wprintf(L"Found! PID=%lu\n", pid); found = TRUE; break;
        }
    } while (Process32NextW(snap, &pe));
}
```

Data Obfuscation (C) – Example

```c
uint32_t djb2_hash_wide(const wchar_t* str) {
    wchar_t c; uint32_t hash = 5381;
    while ((c = *str++)) { hash = ((hash << 5) + hash) + (uint32_t)c; }
    return hash;
}
// [...]
    if (Process32FirstW(snap, &pe)) {
        do {
            DWORD pid = pe.th32ProcessID;
            uint32_t proc_djb2hash = djb2_hash_wide(pe.szExeFile);
            wprintf(L"%6lu  %s\n", pid, pe.szExeFile);
            if (153264797 == proc_djb2hash) {
                wprintf(L"Found! PID=%lu\n", pid);
                found = TRUE;
                break;
            }
        } while (Process32NextW(snap, &pe));
    }
// [...]
```

Mixed Boolean-Arithmetic (MBA) Expressions

MBA expressions replace simple arithmetic with algebraically equivalent Boolean-Arithmetic identities [ZMGJ07]

For example:

1. $a + b = (a \oplus b) + 2\,(a \wedge b)$
   - Arithmetic ops combine **linear**ly with Boolean ops
2. $a \times b = ((a \oplus b) + (a \wedge b))^2 - (a \oplus b)$
   - Products of two variable-dependent subexpressions
   - It is **non-linear**: one variable affects another multiplicatively

Linear vs. Non-linear MBA

Non-linear MBA introduces higher-degree relationships that cannot be reduced with a single linear combination of bits, making them much harder to simplify w.r.t. Linear MBA $\implies$ active research area

- obfuscation $\rightarrow$ creating MBA expressions
- deobfuscation $\rightarrow$ detecting/breaking MBA expressions

Encode Branches

Replace (encoding) unconditional branch instructions with more complex, indirect control-flow mechanisms (like branch functions or push-ret tricks)

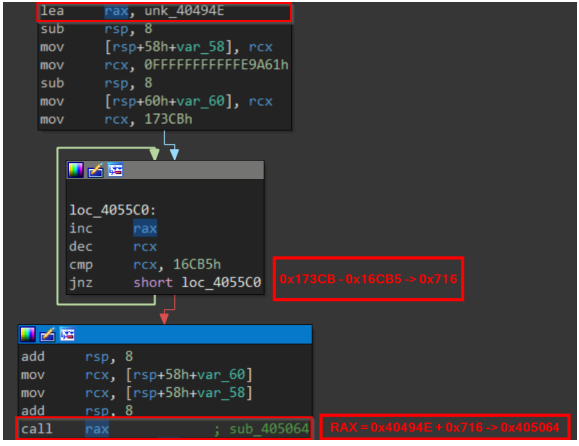
```
    mov     eax, [enc_target]    ; eax = encoded address (DATA)
    xor     eax, 0xA5A5A5A5      ; simple decode step
    call    dispatch

dispatch:
    push    eax
    ret

enc_target:
    dd      (OFFSET L_target) xor 0xA5A5A5A5

L_target:
    ; ... target block ...
```

Data or Control Flow?

The obfuscated **data** is the target branch address, but this also hides an edge in the IPCFG

Opaque constant (and control-flow hiding) example

In a sample of the *Azov* ransomware, the direct call to 0x405064 has been replaced by an indirect one:



https://research.checkpoint.com/2022/pulling-the-curtains-on-azov-ransomware-not-a-skidsware-but-polymorphic-wiper/

Outline

Control-flow obfuscation

Main techniques are:

- *Junk code* insertion
  - Irrelevant instructions or functions
  - Irrelevant branches $\leftrightarrow$ Opaque predicates
  - Bogus function arguments
- Runtime code generation
  - Just-In-Time (JIT) compilation
  - Self Modifying Code (SMC)
- Function
  - Split
  - Merge
  - Inline
- Control-flow flattening
- Virtualization

Example of introducing irrelevant code

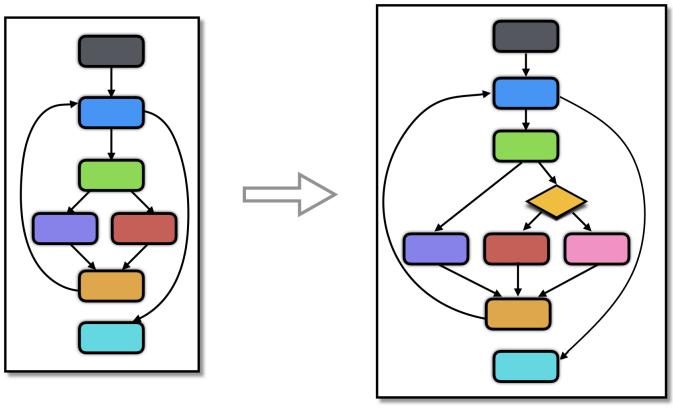
Useless `sub`/`add` instructions in *Azov* ransomware:

```
and     rdx, 0
mov     edx, [rax]
mov     rax, [rbp+moduleBase]
sub     rax, 79C72h
add     rdx, rax
add     rdx, 79C72h
add     rax, 79C72h
mov     [rbp+addressOfNames], rdx
mov     rcx, [rbp+exportDirectory]
add     rcx, _IMAGE_EXPORT_DIRECTORY.AddressOfFunctions
xor     rdx, rdx
mov     edx, [rcx]
sub     rax, 1C5Eh
add     rdx, rax
add     rdx, 1C5Eh
add     rax, 1C5Eh
mov     [rbp+addressOfFunctions], rdx
```

https://research.checkpoint.com/2022/pulling-the-curtains-on-azov-ransomware-not-a-skidsware-but-polymorphic-wiper/

Opaque predicates

A logical expression whose outcome (true or false) is predetermined but deliberately made to look complex, so it must be evaluated at runtime



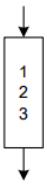
from `https://tigress.wtf/addOpaque.html`

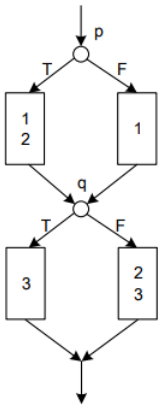
Opaque predicate example

```
static const uint32_t sbox[] = {
    0x9e3779b9u, 0x7f4a7c15u, 0xc6ef3720u, 0x13a5d9f3u,
    0x4f1bbcdc, 0xa3b9d8e1u, 0x6d2f8a21u, 0x5b7e3c09u };

int opaque_predicate(void) {
    uint32_t acc = 0xfeedbeefu;
    for (size_t i = 0; i < sizeof(sbox)/sizeof(sbox[0]); ++i) {
        acc ^= (sbox[i] + (uint32_t)i * 2654435761u);
        acc = (acc << 13) | (acc >> 19);
        acc += (acc >> 7) ^ (acc << 5);
    }
    return (acc & 1u); // return 0
}

int main(void) {
    if (opaque_predicate()) { puts("Dummy code"); }
    else { puts("Real code"); }
    return 0;
```

Dynamic opaque predicates

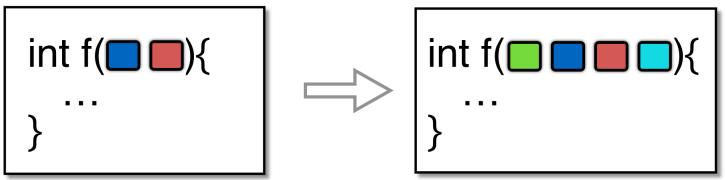
Dynamic opaque predicates consists in using sets of correlated predicates, whose values is the same in one execution, but may change in another





Bogus function arguments

Add extra bogus arguments. Does not work with functions with varargs.



Dynamic code generation and execution

The real code is not present statically but it is generated at runtime (e.g. decrypted, unpacked, JIT-compiled) and then executed from memory.

- **Indicators:**
  - Allocation of memory (e.g. `VirtualAlloc`, `mmap`) and subsequent `VirtualProtect`/`mprotect` calls to set `RWX`/`RX` pages
  - Presence of encrypted/packed blobs, unusual reads from resources, or runtime decryption loops
  - Short-lived threads/regions with executable permissions
  - Atypical call targets
- **Detection challenges:**
  - Benign JITs and legitimate dynamic loaders use similar APIs
  - Do not trust the static memory permissions
    - https://medium.com/@simone.aonzo/the-importance-of-data-execution-prevention-in-malware-analysis-fd29d0c9e03e

Self Modifying Code (SMC)

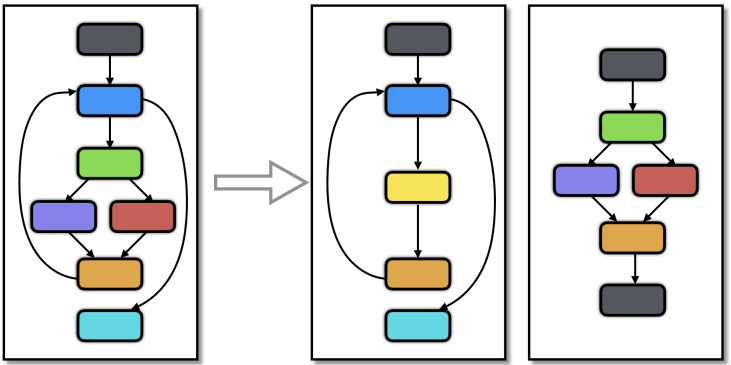
Code that alters its own instructions at runtime (in-place modification)

Dynamic code generation and execution – Example

The listing allocates writable memory, writes x86 machine code (NOP sled + `ret`), flips memory permissions to executable and calls it

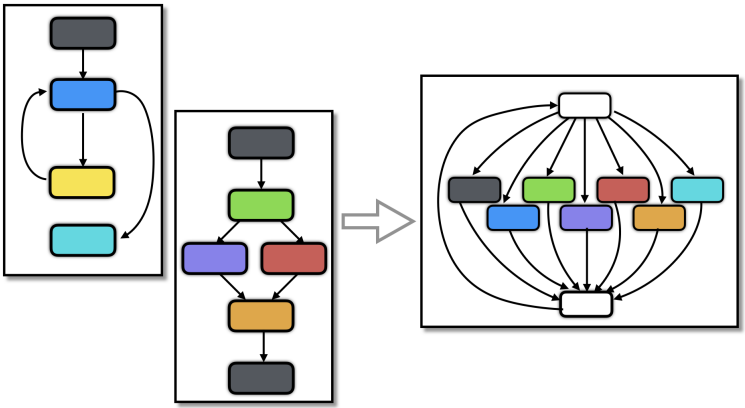
```
void dynamic_nop_execution(int n) {
    size_t sz = (size_t) n * sizeof(unsigned char);
    void* buf = VirtualAlloc(NULL, sz,
                             MEM_COMMIT|MEM_RESERVE, PAGE_READWRITE);
    if (buf == NULL) { exit(1); }
    char* ipbuff = (char*)buf;
    for (size_t i = 0; i < sz; ++i) { ipbuff[i] = 0x90; } // x86 nop
    ipbuff[sz-1] = 0xc3; // x86 ret
    DWORD oldProtect = 0;
    if (!VirtualProtect(buf, sz, PAGE_EXECUTE_READ, &oldProtect)) {
        VirtualFree(buf, 0, MEM_RELEASE); exit(1); }
    typedef void (*nops_fun)();
    nops_fun nops = (nops_fun)buf;
    nops(); // executing the nop sled
    VirtualFree(buf, 0, MEM_RELEASE);
}
```

Function split



from `https://tigress.wtf/split.html`

Function merge (1/3)



from `https://tigress.wtf/split.html`

Function merge (2/3)

```
int foo(int x) { return x*7; }

void bar(int x, int z) {
  if(x==z)
    printf("%d\n", x);
}

int main() {
  int y = 6;
  y = foo(y);
  bar(y, 42);
}
```

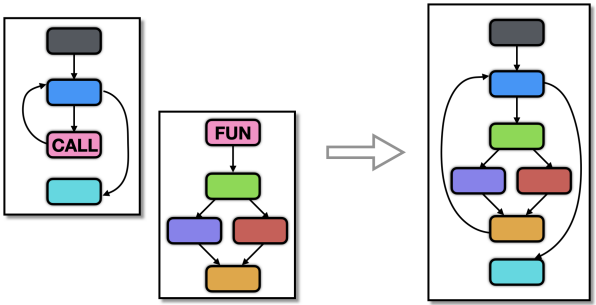
could be transformed into...

Function merge (3/3)

```
int foobar(int x, int z, int s)
{
  if(s==1)
    return x*7;
  else if(s==2)
    if(x==z)
      printf("%d\n", x);
  return 0;
}

int main()
{
  int y = 6;
  y = foobar(y, 99, 1);
  foobar(y, 42, 2);
}
```

Function inlining AKA inline expansion

Replaces a function call with the body of the called function.

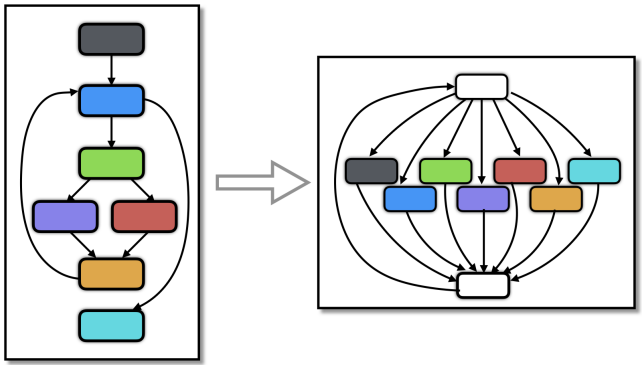


from `https://tigress.wtf/inline.html`

Not only obfuscation

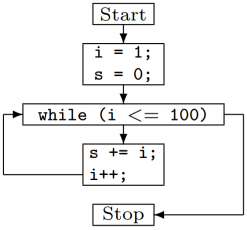
This is also a compiler optimization, with complex effects on performance.
As a rule of thumb, it improves speed at cost of space.

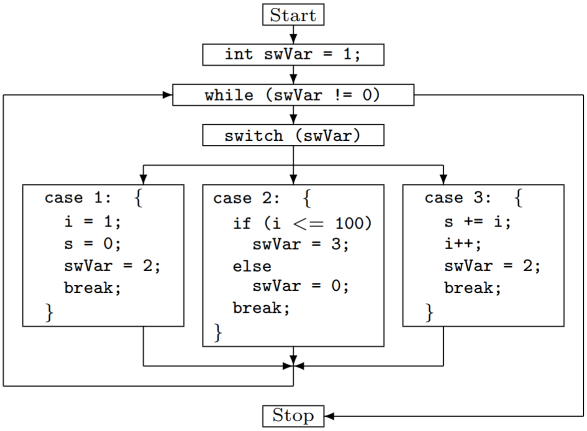
Control-flow flattening (1/2)



from `https://tigress.wtf/flatten.html`

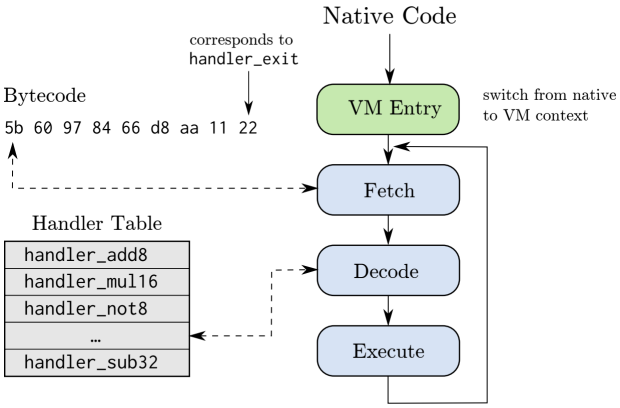
Control-flow flattening (2/2)





from [LK09]

Virtualization



from [BCAH17]

https://tigress.wtf/virtualize.html

Outline

Disassemblers assumptions

Disassemblers usually assume that:

1. In the memory regions dedicated to code, bytes form a *linear* sequence of valid instructions
2. Instruction bytes are consumed exactly once (no overlap)
3. Function *calls/returns* follow a standard calling/ABI convention

Valid assumptions with standard software... but they are not enforced 😈

$\Longrightarrow$ Anti-Disassembly techniques exploit these false assumptions

Anti-Disassembly Taxonomy

1. **Junk bytes (data mixed with code)**
   Bytes inserted between executed instructions that are never reached at runtime but look like instructions to a linear disassembler
2. **Overlapping instructions**
   Control transfers target the middle of an encoded instruction, producing an alternate valid stream when decoding from that target
3. **Call / Return abuse**
   Abusing the semantics of instructions for calling (e.g., x86 `call`) and returning (e.g., x86 `ret`) to create ambiguous control-flow and to embed data that will be interpreted differently

ISA matters!

Fixed-width, aligned ISAs (ARM64) are much less susceptible; variable-width encodings (x86, ARM Thumb) are vulnerable.

Junk bytes (data mixed with code) – Example

```
junk_bytes:
        jz L_END
        jnz L_END
        .byte 0x42, 0x42  # invalid instruction
L_END:
        mov eax, 1
        ret
```

Regardless of the Z flag, the two bytes 0x42, 0x42 will never be executed

- they cannot be executed anyway because they do not constitute a valid instruction
- junk_bytes will not be recognized as a function because the false branch of jnz is not a basic block but... data!?

Overlapping instructions – Example

```
overlapping_instructions:
        push rax            # 50
        mov ax, 0x5eb       # 66 b8 (eb 05) = jmp +7  --|
        xor eax, eax        # 31 c0    ^                |
        .byte 0x74, 0xfa    # je -4 ---/                |
        .byte 0xe8          #                           |
        pop rax             # 58  <---------------------/
        mov eax, 1          # b8 01 00 00 00
        ret                 # c3
```

```
50 66 b8 eb 05 31 c0 74 fa e8 58 b8 01 00 00 00 c3
```

1. (50) (66 b8 eb 05) (31 c0) (74 fa) ->
2. (eb 05) -> (58) (b8 01 00 00 00) (c3)

Overlapping instructions: not only for malware [MM16]

Address	Byte	Sequence 1	Sequence 2	Sequence 3
454017	b8			
454018	eb	mov eax,		
454019	07	ebb907eb		
45401a	b9			
45401b	eb		jmp 45402c	
45401c	0f			
45401d	90	seto bl		
45401e	eb			jmp 454028
45401f	08			
454020	fd	or ch, bh		

Figure 2: An example of overlapping instructions from a piece of malware. All three sequences of blocks execute.

Address	Byte	Sequence 1	Sequence 2
3fe9e8	74		
3fe9e9	01	je 3fe9eb	
3fe9ea	f0		
3fe9eb	0f	lock cmpxchg	
...	..	%ecx,	cmpxchg %ecx,
3fe9f1	00	0x35b0(%ebx)	0x35b0(%ebx)

Figure 3: An example of overlapping instructions from libc.

Call / Return abuse – Example

```
call_ret_abuse:
        call $+5                # e8 00 00 00 00
        add qword ptr[rsp], 6   # 48 83 04 24 06
        ret                     # c3
        mov eax, 1              # b8 01 00 00 00
        ret
```

1. `call $+5`
   - Saves the address of "`add qword ptr[rsp], 6`" onto the stack
   - Jumps forward 5 bytes $\implies$ just the next instruction
2. `add qword ptr[rsp], 6`
   - `[rsp] += 6` $\implies$ now `[rsp]` points to `mov eax, 1`
3. `ret`
   - Pops `rsp` into `rip` $\implies$ `rip = [rsp]`
   - Again, it just jumps to the next instruction
4. `mov eax, 1`
5. `ret`
   - This second `ret` is the standard that returns from the function

Problematic disassembly examples [JLH13] (1/2)

The following example will insert a junk byte with a value `0x9A` in order to confuse a linear sweep disassembler.

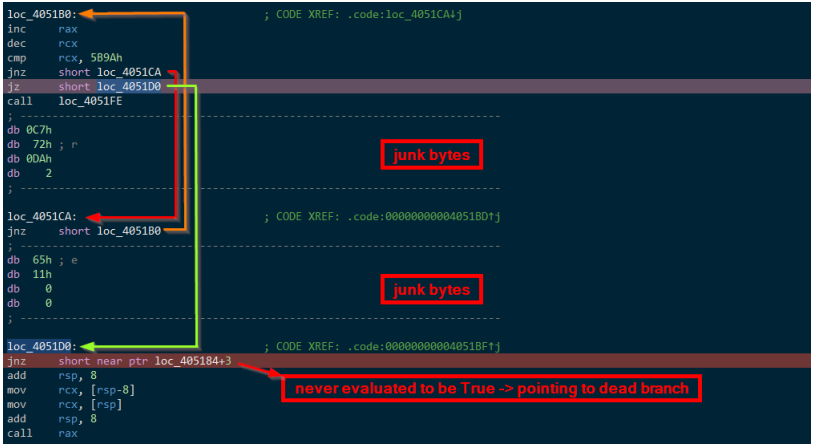
```
JMP foo          EB 03
DB 0x9A          9A   #This is the junk byte
foo:
XOR EAX,EAX      31 C0
XOR EBX,EBX      31 C3
INC EAX          40
INC EBX          43
INT 0x80         CD 80
```

This piece of code would be disassembled in the following way.

```
JMP foo+1                        EB 03
foo:
CALL DWORD 0x4340:0xC331C031     9A 31 C0 31 C3 40 43
INT 0x80                         CD 80
```

Opaque predicates and anti-disassembly

Opaque predicates and anti-disassembly (junk bytes) in *Azov* ransomware:



https://research.checkpoint.com/2022/pulling-the-curtains-on-azov-ransomware-not-a-skidsware-but-polymorphic-wiper/

Outline

Packing

Packers are programs that transform executables

- Originally, used to save (disk) space by compressing programs
- Nowadays, mostly used to thwart static analysis
  - Still used when embedding huge resources
  - Encryption is also used
- Packers are also known as Crypters
  - Focus on stealth rather than compression

When an executable gets packed:

- an unpacking stub is added to the packed program
- whose entry-point points to this stub
- (most) references to external libraries/functions are typically removed

Off-the-shelf packer

A ready-made software product that takes an existing executable and produces a packed version. Famous ones:

- UPX: Cross-platform, open source packer 👑
- ASPack: Advanced commercial packer with a high compression ratio
  - Borderline: it also has protection features
- Old, no more maintained:
  - PEtite: Freeware, similar to ASPack
  - FSG: Freeware, fast to unpack
  - MEW: Specifically designed for small binaries
  - nPack: Freeware, many file types (exe, dll, ocx)
  - MPRESS: Freeware, more complex packer

Prevalence in malware

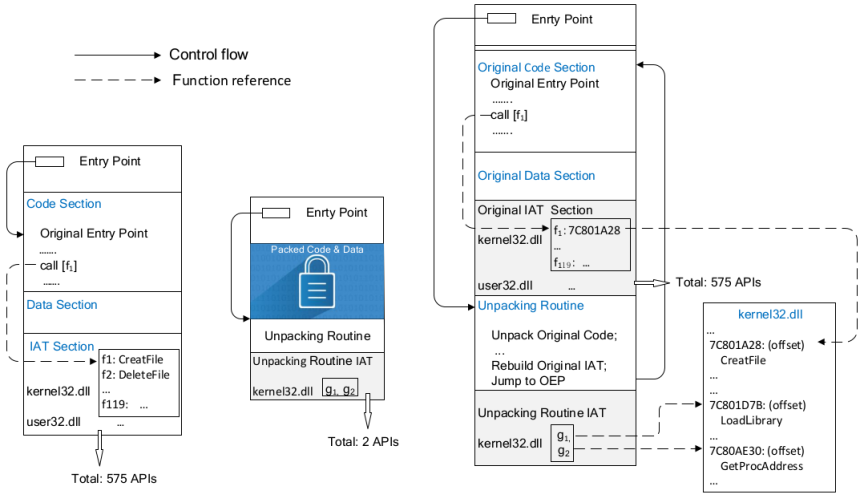
In state-of-the-art malware datasets [VAD+25], about 25% are packed with an off-the-shelf packer.
Among them, about 60% use UPX and 15% ASPack.

Unpacking stub

The unpacking stub is a piece of code that

1. unpacks (=decompress/decrypt) the original program
   - in the same process, or in others to evade detection
   - often, it needs to allocate new memory
2. usually, resolves (the original) imports
   - loading the corresponding libraries if needed
3. jumps to the Original Entry Point (OEP), with the so-called tail-jump

Example: FSG



Taken from [CMF+18]

Indicators of packing

How can we detect a packed executable?

- few imports, in particular `LoadLibrary` and `GetProcAddress`
- abnormal section features, like
  - names, e.g., `UPX0`/`UPX1`
  - sizes; e.g., an empty code section (on disk, with non-zero virtual size)
- entropy is an indicator, but it may be unreliable [MAUP+20]
- off-the-shelf packers introduce known byte patterns
  - Detect It Easy — `https://github.com/horsicq/Detect-It-Easy`

Features [UPBSB15]

- Unpacking **layers**
  - How many times code is unpacked and executed after being written
  - From a **single** layer to many nested layers (**multi-layer**)
- **Transition** model
  - **Linear**: execution moves forward layer by layer
  - **Cyclic**: execution jumps back to previous layers
- Packer **isolation**
  - **Tail jump**: packer runs first, then hands control to the original code
  - **Interleaved**: packer and original code execute mixed together
- Unpacking **frames**
  - **Single-frame**: a whole layer is unpacked at once
  - **Multi-frame**: code is unpacked piece by piece over time
- **Code visibility**
  - **Full**: entire code present in memory at once
  - **Incremental**: unpacked on demand
  - **Shifting**: code is re-packed after execution

Complexity Classes [UPBSB15]

From the least to the most complex:

I) a single unpacking routine is executed before transferring the control to the unpacked program

II) multiple unpacking layers are executed sequentially and lead to the original code at the end

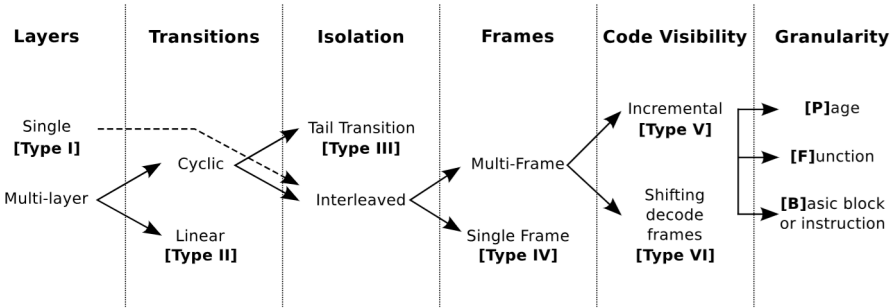
III) intermediate layers are executed in loops

IV) the packer code is interleaved with the execution of the unpacked program

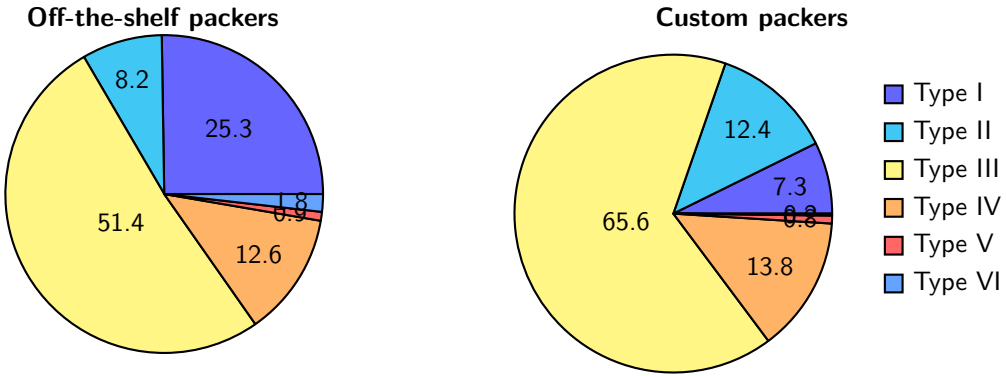
V) pieces of the original program are unpacked on-demand

VI) only a single fragment of the original program (as little as a single instruction) is unpacked in memory at any moment in time

Features and Complexity Classes [UPBSB15]



Distribution [UPBSB15]



Unpacking

Off-the-shelf packers, e.g. UPX, may provide an unpacker.

If an unpacker is not available, we can tackle the problem by

- Reversing the unpacking algorithm, and then writing an unpacker
  - Do some research, maybe someone has already done it
- Running/emulating the program until it is unpacked (in some "jail/sandbox", using VMs, emulators, instrumentation, ...) to
  1. Find the OEP
  2. Dump the unpacked version from memory

  Then, fix/reconstruct the (PE/ELF/...) header. See:
  - *Mal-Unpack*
    `https://github.com/hasherezade/mal_unpack`
  - *Unipacker*
    `https://github.com/unipacker/unipacker`

Finding the OEP and dumping the program

Typically, the unpacking stub

1. Saves the execution-context on the stack
2. Unpacks its payload
3. Resolve the imports
   - An API call from a rebuilt-IAT means that the control flow already reached the OEP [CMF+18]
   - A BP/hook on `GetProcAddress` may allow you to detect the beginning of this phase
4. Restores the original context, by "popping" it from the stack
   - A HW-breakpoint on stack access can help in finding the OEP
5. Jumps to the OEP
   - At the beginning, the area containing the OEP is usually invalid/empty
   - Execution of addresses written by the program can indicate the OEP
     - or another **layer** of packing [UPBSB15]

Because of standard initialization code, programs may

- share the same startup code, which can be searched in memory
- call common functions, e.g. `GetCommandLine`; BPs on those can help

Special cases

If the unpacker is “lazy” and relies on standard API `RtlDecompressBuffer` and/or `CryptDecrypt`:

- BP on those functions
- When BP is hit, get the
  - address of the buffer
  - pointer to the (output) size
- Then execute until return
- Check/dump the memory area

A general approach using a debugger

Breakpoint on

RtlDecompressBuffer	
CryptDecrypt	(see previous slide)
CreateProcess{*,InternalW}	used in many *injection* techniques; however, `ShellExecute` uses flag `CREATE_SUSPENDED`
NtUnmapViewOfSection	used by *process hollowing*
VirtualProtect(Ex)	*hooking* or dynamically-generated code
LocalAlloc/VirtualAlloc(Ex)	the result value can be followed in dump to see if something interesting is written there
NtWriteVirtualMemory	
WriteProcessMemory	to see what gets written, and where
CreateRemoteThread	to start shellcode/`LoadLibrary`
CreateFileTransactedW	may indicate process Doppleganging

Outline

Tamperproofing

Techniques used to hinder, deter or detect unauthorized modification

- Also used for real-world objects and hardware, e.g.:
  - Envelopes, warranty labels, screws
  - Microprocessors of car keys, crypto wallets, smartcards
- In software, it verifies its **integrity**

Are the instructions, their execution flow, and the data on which they operate the original ones?

Tamper "proof" is a misnomer

Any device or system can be foiled by a person with sufficient knowledge, equipment, and time

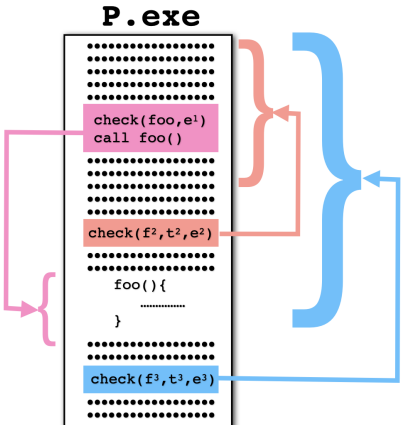
Anti-tampering – How?

Integrity **checkers** are inserted in the code over critical code data regions

- Which? Integrity $\Leftrightarrow$ Cryptographic hash functions
  - For performance reasons, even just checksum
- Where? Unpredictable and unexpected locations
  - Not only at the beginning, but throughout the entire execution
  - Performance tradeoff
  - First anti-tampering, then obfuscation
- Who?
  - Local (same process) and/or remote (server)
  - More privileged process (kernel driver, rootkit, etc.)
    - E.g., Sony BMG copy protection rootkit scandal
  - Hardware assistance
- How to manage violations?
  - Nondeterministic responses
    - e.g., randomly delayed `exit()`
  - Stealth degradation
    - e.g., introduce bugs that degrade the user experience

Checkers example

- `foo()` is checked by the pink checker, prior to being called
- The e^1 value is the expected hash value computed over `foo`
- Can also cover arbitrary levels of (overlapping) regions $f^i - t^i$
  - $e^2 = hash(f^2 - t^2)$ and $e^3 = hash(f^3 - t^3)$



Stealth Degradation in Videogame Industry

Anti-tamper technique that allows play but *subtly degrades* or breaks the experience when piracy/cracking/cheating is detected

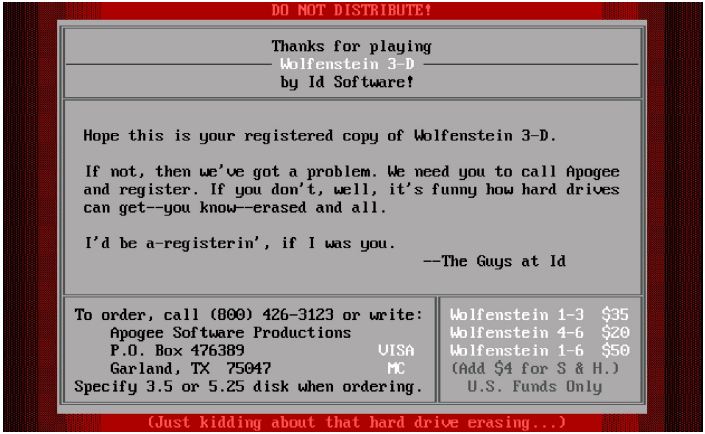
- FADE (DRM system): reversed controls and random crashes
- EarthBound (SNES): dramatic spike in enemy encounters
- The Settlers III (Win): iron smelters produced *pigs* instead of iron 😆
- Michael Jackson (DS): loud and persistent vuvuzela/horn noises 😆
- Spyro: Year of the Dragon (PSX)
  - basic copy protection $\Rightarrow$ "we cracked it" feeling
  - secondary "protection" based on stealth degradation
- **Shadowban**: the user/client is not told they were banned
  - ban = frustrating gaming experience
  - E.g., COD Warzone dedicated servers for cheaters

Psyc warfare: *"cheating or using pirated copies* $\implies$ *frustrating gaming experience"*

Takeaways: Do not provide the attacker with an oracle that can tell them whether they have succeeded or failed.

The best defense is a good offense

Wolfenstein 3D (1992) by id Software did not need DRM; it had threats.
Even if the “aggressive” protection mechanism was a joke... it definitely made you think twice 😅



Outline

Watermarking

Covertly embed an **identifier** (owner, build ID, licensee) into a program
$\Longrightarrow$ there is a different version of the program for each identifier

- Preserves **semantics**: no change in observable behavior
- **Low overhead**: minimal time/space/runtime impact
- **Stealth**: hard to find
- **Robust**: hard to remove/forge
- Supports **ownership claims** and **leak attribution**

The main deployment of the digital watermarking is into noise-tolerant signals such as audio, video or images.

Watermarks Taxonomy

Watermarks differ by how the identifier is "carried"

- **Static**: embedded in code or data structures
  - E.g., code: CFG patterns, basic block order, Mixed Boolean-Arithmetic
  - E.g., data structures : arrays, graphs, trees whose shapes encode bits
- **Dynamic**: materialize only at runtime
  - E.g., heap or call-stack structures

Carriers are often *combined* and made *redundant* for robustness
They are protected by a combination of obfuscation and tamperproofing.

Watermarking – Example

I have two customers. I distribute my software (x86-64 Linux binaries) with two semantically equivalent versions of the Absolute Value function.

```
absval1:
  mov eax, edi    ; eax = x
  test eax, eax   ; check sign
  jge done1       ; jump if >= 0
  neg eax         ; eax = -eax
done1:
  ret
```

```
absval2:
  mov eax, edi   ; eax = arg1
  cdq            ; eax sign -> edx
  xor eax, edx   ; eax ^= sign
  sub eax, edx   ; eax -= sign
  ret
```

If I notice that my software has been cracked and distributed illegally, I can understand where the leak originated.

Outline

Evasion

Evasive techniques are used to prevent the analysis of the app's behavior

1. **Conditional execution**
   - The Runtime Environment and Analysis Component introduce **artifacts**
   - E.g., VMWare NIC MAC addresses starts with `00:05:69`
   - In this context they are called **red pills**
   - If red pills $\implies$ under analysis $\implies$ behave differently
2. **System Tampering**
   - Neutralize the Analysis Component
   - E.g., uninstalling the AntiVirus
   - Very effective, but also very "noisy"
3. **Stealth execution**, two types:
   1. Action Hiding
      - Make the activity fly under the radar of the Analysis Component
      - E.g., direct/indirect syscalls to bypass hooks
   2. Origin Hiding
      - Use another (usually trusted) process to perform the actions
      - E.g., Process injection or Powershell command

Red and Blue pills

- **Red pills**: Artifacts that expose the presence of an analysis system
  - Searched for by surreptitious software
    - AKA **split personality** [BCK+10]
- **Blue pills**: Countermeasures that conceal the analysis system
  - Intentionally introduced to trigger real behavior of the target software

Outline

Anti-Debugging techniques

General approaches to detect the presence of a debugger:

- Breakpoints can be detected
- Signals/Exception are handled differently
- Time delays can be measured
  - Timing will be discussed later...
- The presence of known debuggers processes can be detected

Evasions/Attacks:

- Callback executions are "hidden", if you don't pay attention to functions registering them
- A process can register itself or a child as its own debugger, to prevent the attachment of another
  - or detect the fact that one is already attached
- A process can hijack the mouse/keyboard/screen, to prevent user-interactions

Debug flags

Flags in system tables indicate that a process is being debugged. These can be read by using API functions or examining memory

Windows API

- `kernel32!IsDebuggerPresent`
- `kernel32!CheckRemoteDebuggerPresent`
- ...

Manual checks, by reading:

- `BeingDebugged`, byte at offset 2 in the PEB (PEB = `fs:[0x30]`)
- `NtGlobalFlag` in the PEB
- Heap flags
- ...

`https://www.nirsoft.net/kernel_struct/vista/TEB.html`

`https://www.nirsoft.net/kernel_struct/vista/PEB.html`

Linux

- API: `ptrace`
- Manual checks: `TracerPid` in `/proc/self/status`

Parent processes

Another simple check consists in verifying the parent-name/presence of "suspicious" windows; e.g. the names of known debugging tools

Windows `user32!FindWindow(Ex)`, `user32!EnumWindows`, `user32!EnumThreadWindows`, ...

Linux You can get the parent via `getppid`, and then check `Name` in `/proc/`*`parent-pid`*`/status`

To thwart string analysis, known-names can be hashed/encrypted

Exceptions are handled differently

- `kernel32!CloseHandle`/`ntdll!NtClose` raise an exception when an invalid handle is passed *and* the process is being debugged

  `https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntclose`
- `kernel32!SetUnhandledExceptionFilter` allows an application to replace the default handler; however, the handler is not called is the process is being debugged

  `https://docs.microsoft.com/en-us/windows/win32/api/errhandlingapi/nf-errhandlingapi-setunhandledexceptionfilter`
- `kernel32!RaiseException` can be used to raise exceptions consumed by a debugger, such as `DBC_CONTROL_C`

  `https://docs.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-exception_record`

Unix Signals

Similar idea, with signals:

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>

void handler(int signo)
{
        printf("Not debugged!\n");
        exit(0);
}

int main()
{
        signal(SIGTRAP, handler);
        asm("int3");
        printf("Debugged...\n");
}
```

Self-debugging

Linux `ptrace(PTRACE_TRACEME, ...)`:

- if it fails, there is a debugger
- OTOH if it succeeds twice (or when called with bad arguments), then there are probably both a debugger *and* `ptrace` has been hooked
  - A program may check `LD_PRELOAD` too

Windows You can create a child process that tries to debug its parent

Breakpoints

Software Breakpoints can

- be searched/removed in
  - pre-determined memory ranges (e.g., function bodies)
  - runtime locations
    - e.g., a function can retrieve its saved return address and look for a BP at that instruction
- if code is copied to newly allocated memory and run there
  - the program crash: it is not the same memory region where the debugger expects to handle it

Hardware Breakpoints use CPU debug registers (e.g., `DR0`–`DR3` on x86)

- Windows: `GetThreadContext()` retrieve a thread's CPU context
  - that includes all general-purpose registers and the debug registers
- Linux: you cannot do the same, but still rely on `ptrace` tricks

Mitigations

- Skipping/patching the checks
- Flags in user-space (e.g. `BeingDebugged`) can be altered
- Functions that invoke system-calls can be *intercepted*
  - breakpoints
  - hooking
  - ...

Anti anti-debug: ScyllaHide

ScyllaHide, `https://github.com/x64dbg/ScyllaHide`, can automatically defeat most anti-debugging techniques

Outline

Anti-Instrumentation & Anti-Hooking

Instrumentation

- Observing or modifying a program's behavior by inserting extra code
- *Static*: modify binary before execution
- *Dynamic*: inject at runtime (e.g., DBIs like DynamoRIO and Frida)

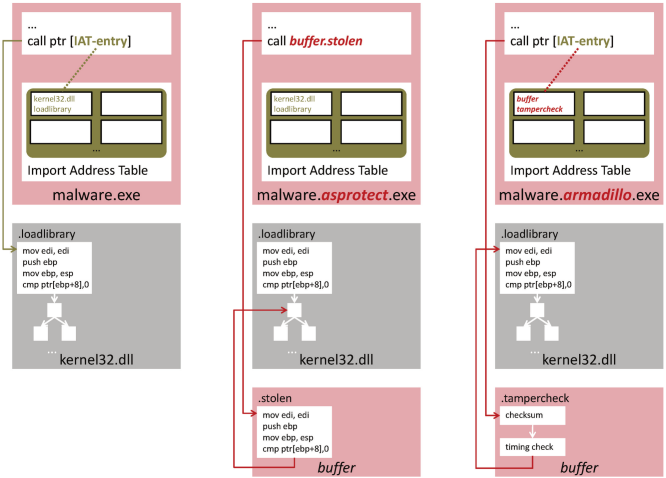
Hooking is a dynamic instrumentation technique

- Diverting calls/flows (hook) to custom code (callback)
- *Common techniques*:
  - Change import address table entries (Win IAT, Linux GOT+PLT)
  - Inline patching. E.g., overwrite function prologue with a jump

Anti-(Instrumentation|Hooking)

- Anti-tampering first and foremost
- Searching loaded modules/libraries for known DBIs
- Load a new copy (or some bytes AKA *Stolen bytes*) of ext. libraries
- ...

Stolen Bytes, an Anti-hooking Technique



Taken from [RM13]

Outline

Environment detection

Sandboxes and other analysis environment can be detected by checking

- the presence of a special processes or loaded DLLs
- the number of running processes
- installed programs/their absence
- the presence of an hypervisor
- RAM size/screen resolution/...
- mouse movement, keyboard activity, ...
- ...

Top open-source project with example code for detection:

al-khaser `https://github.com/LordNoteworthy/al-khaser`

Example: red pills in a VMware virtual machine

- Virtual network interfaces can have specific MACs
  - e.g., `00-50-56-...`, `00-0C-29-...`, ... See, e.g., `https://hwaddress.com/company/vmware-inc/`
- Display device names may contain the string `"VMware"`
- Drivers, found in the
  - registry: `HKEY_LOCAL_MACHINE\System\CurrentControlSet\Control\Class\{4D36E968-E325-11CE-BFC1-08002BE10318}\0000\DriverDesc`
  - file system: `%WINDIR%\system32\drivers\vmmouse.sys`
- `CPUID` x86 instruction with `EAX=0`
  - Returns the CPU's manufacturer 12 characters ID string
  - Stored in `EBX`, `EDX`, `ECX`
  - "`VMwareVMware`"
- ...

Outline

Timing

General algorithm:

1. Get the time with fine granularity
   - Internal – e.g.:
     - `RDTSC` x86 instruction
     - APIs: Windows `GetTickCount()` or Linux `clock_gettime()`
   - External – e.g.:
     - NTP protocol
     - `https://gettimeapi.dev/v1/time?timezone=UTC`
2. Do "something" that you know takes a certain amount of time – e.g.:
   - `sleep(300)` stop execution for 5 minutes
   - `cpuid` instruction (slower in a VM than on bare metal)
3. Get the time again and compute the elapsed interval
4. (optional) Repeat from ① to calculate an average

Timing in Locky ransomware

```
BOOL rdtsc_diff_locky()
{
   unsigned __int64 tsc1, tsc2, tsc3;
   for (int i = 0; i < 8; i++) { // 8 times in case of fluctuations
      tsc1 = __rdtsc();
      GetProcessHeap(); tsc2 = __rdtsc();

      // CloseHandle() slower (~x10) than GetProcessHeap() on bare metal
      CloseHandle(0); tsc3 = __rdtsc();

      // Did it take at least 10 times more to perform
      // CloseHandle(0) than it took to perform GetProcessHeap() ?
      if ((tsc3-tsc2) / (tsc2-tsc1) >= 10) return FALSE;
   }
   return TRUE; // Consistent small ratio of differences => VM
}
```

On bare metal, `CloseHandle()` takes longer than `GetProcessHeap()`.
In a virtualized environment, hypervisor interceptions slow both calls uniformly, shrinking the timing gap.

References I

[BCAH17] Tim Blazytko, Moritz Contag, Cornelius Aschermann, and Thorsten Holz.
Syntia: Synthesizing the semantics of obfuscated code.
In *26th USENIX Security Symposium (USENIX Security 17)*, pages 643–659, 2017.

[BCK+10] Davide Balzarotti, Marco Cova, Christoph Karlberger, Engin Kirda, Christopher Kruegel, and Giovanni Vigna.
Efficient detection of split personalities in malware.
In *NDSS*, 2010.

[CMF+18] Binlin Cheng, Jiang Ming, Jianmin Fu, Guojun Peng, Ting Chen, Xiaosong Zhang, and Jean-Yves Marion.
Towards paving the way for large-scale windows malware analysis: Generic binary unpacking with orders-of-magnitude performance boost.

References II

In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 395–411, 2018.

[JLH13] Christopher Jamthagen, Patrik Lantz, and Martin Hell.
A new instruction overlapping technique for anti-disassembly and obfuscation of x86 binaries.
In *Anti-malware Testing Research (WATeR), 2013 Workshop on*, pages 1–9. IEEE, 2013.

[LK09] Tımea László and Ákos Kiss.
Obfuscating c++ programs via control flow flattening.
Annales Universitatis Scientarum Budapestinensis de Rolando Eötvös Nominatae, Sectio Computatorica, 30(1):3–19, 2009.

References III

[MAUP+20] Alessandro Mantovani, Simone Aonzo, Xabier Ugarte-Pedrero, Alessio Merlo, and Davide Balzarotti.
Prevalence and Impact of Low-Entropy Packing Schemes in the Malware Ecosystem.
In *NDSS*, 2020.

[MM16] Xiaozhu Meng and Barton P Miller.
Binary code is not easy.
In *Proceedings of the 25th International Symposium on Software Testing and Analysis*, pages 24–35. ACM, 2016.

[NC09] Jasvir Nagra and Christian Collberg.
Surreptitious software: obfuscation, watermarking, and tamperproofing for software protection.
Pearson Education, 2009.

References IV

[RM13] Kevin A Roundy and Barton P Miller.
Binary-code obfuscations in prevalent packer tools.
ACM Computing Surveys (CSUR), 46(1):1–32, 2013.

[UPBSB15] Xabier Ugarte-Pedrero, Davide Balzarotti, Igor Santos, and Pablo G Bringas.
SoK: Deep packer inspection: A longitudinal study of the complexity of run-time packers.
In *2015 IEEE Symposium on Security and Privacy*, pages 659–673. IEEE, 2015.

[VAD+25] Antonino Vitale, Simone Aonzo, Savino Dambra, Nanda Rani, Lorenzo Ippolito, Platon Kotzias, Juan Caballero, and Davide Balzarotti.
The polymorphism maze: Understanding diversities and similarities in malware families.
In *European Symposium on Research in Computer Security*, pages 505–525. Springer, 2025.

References V

[ZMGJ07] Yongxin Zhou, Alec Main, Yuan X. Gu, and Harold Johnson.
Information Hiding in Software with Mixed Boolean-Arithmetic Transforms.
In *Proceedings of the 8th International Conference on Information Security Applications*, WISA'07, page 61–75, Berlin, Heidelberg, 2007. Springer-Verlag.